

CNN Fusion optimization

Mr. Pranay Arvind Patil
IIT Gandhinagar, India
pranay.patil@iitgn.ac.in

Prof. Joycee Mekie
IIT Gandhinagar, India
joycee@iitgn.ac.in

Abstract—Convolutional Neural Networks (CNNs) are widely used for diverse applications, and many successful attempts have been made to improve their efficiency and computational latency by utilizing custom hardware accelerators and optimizing the computational mapping to the hardware. However, there is still scope for further improvements as novel custom hardware and better optimizers are continuously being developed. The mapping of fused-layer CNNs has been explored on custom hardware accelerators, but these methods typically take a long time to find the optimal mapping. In this work, we use analytical methods to constrain the design space exploration, along with an existing modeler, to find an optimal solution much faster. We have successfully reduced the search time by 16-20x compared to DeepFrack, while maintaining a comparable energy consumption.

Index Terms—CNN, Fused layer mapping, optimizer, design space exploration.

I. INTRODUCTION

Convolutional Neural Networks (CNNs) are extensively used in applications such as computer vision, natural language processing, and speech recognition. Due to their widespread use and the increasing size of modern models, the computational and memory demands of CNNs have grown. To address these demands, specialized hardware accelerators are routinely deployed. However, hardware alone is insufficient; the way neural network computations are mapped and scheduled onto this hardware affects the overall latency and energy efficiency.

A promising approach to further optimize performance is layer fusion, where operations from multiple CNN layers are computed together to minimize expensive off-chip memory accesses. While the mapping of fused-layer CNNs has been explored on custom hardware accelerators, the design space of possible mappings is massive. Consequently, state-of-the-art frameworks (such as DeepFrack) take a prohibitively long time to perform the design space exploration (DSE) to find an optimal mapping.

In this work, we propose a novel approach to accelerate the optimization process for fused-layer CNN mappings. By leveraging analytical methods, we constrain the design space exploration, effectively pruning improbable mappings early in the process. We integrate these analytical constraints with an existing modeler to rapidly identify the optimal hardware mapping.

Our main contributions are as follows:

- We introduce an analytical framework to bound the design space for fused-layer CNN mappings, significantly reducing the number of evaluated configurations.

- We demonstrate a 16–20 $\times$ reduction in optimization time compared to the DeepFrack baseline.
- We verify that the mappings generated by our accelerated search achieve energy consumption profiles that are comparable to those found by the exhaustive baseline.

II. METHODOLOGY

Fusion of CNN layers is mainly done to avoid the transfer of data between the DRAM and caches. A pyramid scheme for fusion is applied, where a large input tile is taken for a given layer and the convolution is done with the corresponding kernel to get the output. This output tile will then become the input for the next layer. We continue to go across the layers like this. This reduces the number of DRAM accesses performed across the chip. Therefore, less energy is consumed, unlike in the traditional method where we evaluate a single layer entirely, send the data to the DRAM, and after that bring it back to the caches for computation of the next layer.

As we go across the layers in fusion, the size of the tile reduces. This is due to not having the complete data at the edges, so we can't compute the edges of the output tiles. This is why it is called pyramid fusion, where the base is the input and the top is the output tile.

Now, as we go across the tiles in the output and project the required input for that, we will find there is an overlap on the input side. This is referred to as the halo. Because of this, certain decisions need to be made about whether the halo for the intermediate data (e.g., layer 2 has data that overlaps for 2 tiles of layer 5) should be recomputed or preserved in the cache.

Therefore, we need to make the following decisions:

- 1) We need to decide on the preservation of data on the cache or if we should recompute it.
- 2) We need to decide the start and end layer for a given fusion (say, layer 1 to 3 are fused, and after that, layer 4 to 8 are fused).
- 3) Then we need to determine the size of the tile. For the given start and end layer, we need to find the optimal tile size.

We are determining the data tile size using analytical methods such that we want the entire fusion layer to remain in the cache, as DRAM accesses are very expensive.

After that, we calculate the energy and latency for a given start and end layer for all possible combinations using a modeler. Then, this data is used to determine the ideal mapping for fusion. This isn't a brute force approach, so it can miss

out on certain edge cases, but it ends up pruning the design space just enough that we get an almost ideal mapping without having a large exploration time.

To formalize this, our analytical tiler (Algorithm 1) relies on specific hardware capacities and loop tiling variables. In our formulations, *capacities.peInput* and *capacities.peOutput* represent the storage capacities of a single Processing Element's (PE) local input and output buffers, respectively. The dimensions being tiled are denoted as follows: T_{out} is the spatial width and height of the output tile, T_{in} is the required spatial dimension of the input tile, T_c is the tiled number of input channels, and T_m is the tiled number of output channels (filters).

Algorithm 1 Polyfuse Analytical Tiler for Fused CNN Layers

Result: Valid tile dimensions for each layer or null if infeasible

```

1: Function evaluateStack(layers, capacities):
2:  $high \leftarrow \min_{l \in layers} (l.P)$ 
3:  $low \leftarrow 1$ ,  $bestT_{out} \leftarrow 0$ 
4: while  $low \leq high$  do
5:    $mid \leftarrow \lfloor (low + high)/2 \rfloor$ 
6:   if checkConstraints(layers, mid, capacities) then
7:      $bestT_{out} \leftarrow mid$ 
8:      $low \leftarrow mid + 1$ 
9:   else
10:     $high \leftarrow mid - 1$ 
11:   end if
12: end while
13: if  $bestT_{out} \leq 0$  then
14:   return null
15: end if
16:  $tileSizes \leftarrow \emptyset$ 
17:  $currentT_{out} \leftarrow bestT_{out}$ 
18: for  $i \leftarrow \text{length}(layers)$  down to 1 do
19:    $l \leftarrow layers[i]$ 
20:    $T_{in} \leftarrow (currentT_{out} - 1) \times l.stride + l.R$ 
21:    $maxT_m \leftarrow \max(1, \lfloor capacities.peOutput/currentT_{out}^2 \rfloor)$ 
22:    $T_m \leftarrow \min(l.M, maxT_m)$ 
23:    $maxT_c \leftarrow \max(1, \lfloor capacities.peInput/T_{in}^2 \rfloor)$ 
24:    $T_c \leftarrow \min(l.C, maxT_c)$ 
25:    $tileSizes[l] \leftarrow (currentT_{out}, T_{in}, T_c, T_m)$ 
26:    $currentT_{out} \leftarrow T_{in}$ 
27: end for
28: return tileSizes

```

III. IMPLEMENTATION DETAILS

The optimizer is built using python, and it is integrated with timeloop+accelergy V4. LoopTree is used as the modeler to find the energies and latency of a given fusion stack. The optimizer isn't completely hardware agnostic yet, but it can parse through yaml files of a given architecture in timeloop syntax, to provide the mappings:

- **Hardware Abstraction Layer (HAL):** We dynamically parse through the architecture, problems and the con-

Algorithm 2 Polyfuse Spatial Router

Result: Spatial PE allocation (X_i, Y_i) for each layer i in a fused stack

```

1: Function optimizeSpatialRouting(layers, HW):
2:  $N \leftarrow \text{length}(layers)$ 
3: if  $N \leq 1$  then
4:   return  $[(0, 0)] \times N$ 
5: end if
6:  $X_{max} \leftarrow HW.peX - 1$ ,  $Y_{max} \leftarrow HW.peY - 1$ 
7: Objective: minimize  $\sum_{i=1}^{N-1} ((X_{i+1} - X_i)^2 + (Y_{i+1} - Y_i)^2)$ 
8: Subject to bounds:  $0 \leq X_i \leq X_{max}$ ,  $0 \leq Y_i \leq Y_{max}$ 
9: Subject to constraints: Extreme vector causality
10:  $initialGuess \leftarrow (X_{max}/2, Y_{max}/2)$  for all  $i$ 
11:  $res \leftarrow \text{SLSQP\_Minimize}(\text{Objective}, initialGuess,$ 
12:   bounds, constraints)
13: if  $res.success$  then
14:    $routing \leftarrow \text{round}(res.x)$ 
15: else
16:    $routing \leftarrow \text{fallback linear mapping}$ 
17: end if
18: return routing

```

Algorithm 3 Polyfuse Dynamic Programming Partitioner

Result: Optimal partitioned layer stacks and minimum total cost

```

1: Function partition(layers):
2:  $N \leftarrow \text{length}(layers)$ 
3:  $dp \leftarrow [\infty] \times (N + 1)$ 
4:  $dp[0] \leftarrow 0$ 
5:  $parent \leftarrow [-1] \times (N + 1)$ 
6: for  $i \leftarrow 1$  to  $N$  do
7:   for  $j \leftarrow 0$  to  $i - 1$  do
8:      $stack \leftarrow layers[j \dots i - 1]$ 
9:      $isValid, tileSizes \leftarrow \text{evaluateStack}(stack, HW)$ 
10:    if not  $isValid$  then
11:      continue
12:    end if
13:     $baseEnergy \leftarrow \text{evaluateSingleBlock}(stack, tileSizes, HW)$ 
14:    if  $baseEnergy == \infty$  then
15:      continue
16:    end if
17:     $routing \leftarrow \text{optimizeSpatialRouting}(stack, HW)$ 
18:     $routingCost \leftarrow \text{calcRoutingCost}(routing)$ 
19:     $totalCost \leftarrow baseEnergy + routingCost$ 
20:    if  $dp[j] + totalCost < dp[i]$  then
21:       $dp[i] \leftarrow dp[j] + totalCost$ 
22:       $parent[i] \leftarrow j$ 
23:    end if
24:  end for
25: end for
26: if  $dp[N] == \infty$  then
27:   return null
28: end if
29:
30: return reconstructOptimalPartition(parent)

```

straint files to extract the relevant information for the optimizer. From the architecture files we extract the size of the caches and classify them according to what datatype it stores, which is found using the constraints. Along with cache sizes, number of PE's is also extracted. From the problems size of the convolution is obtained. We use the constraint which is fed into LoopTree for faster evaluation.

- **Optimization Solvers:** The framework relies on standard mathematical libraries to implement the optimization stages. The spatial router (Algorithm 2) utilizes the Sequential Least Squares Programming (SLSQP) optimizer from the SciPy library to minimize the sum of squared distances for PE mapping. The dynamic programming partitioner (Algorithm 3) is implemented efficiently using 1D memoization arrays to ensure an $O(N^2)$ time complexity for exploring all valid contiguous layer sub-stacks.
- **Timeloop, LoopTree, and Accelergy Integration:** To ensure rigorous and scientifically valid energy and latency evaluations, our framework integrates directly with both the Timeloop mapper (for single-layer execution) and LoopTree (for fused multi-layer blocks). When a valid fused stack is identified by the DP partitioner, the framework generates the corresponding mapped workload and invokes LoopTree alongside Accelergy as the back-end evaluator. This guarantees that the reported energy profiles for layer fusion accurately reflect the underlying physical hardware constraints.

IV. ALTERNATE APPROACHES EXPLORED

The problem of optimizing the nested for loops is a well explored problem in compiler theory. Where works like Pluto have tried to optimize them by modelling an n times nested for loop as an n dimension polyhedra that would be transformed and a schedule would be chosen such that a given cost function is minized. These approaches have worked very well for multithreaded cpu's and gpu's.

By changing the cost function an optimal mapping could also be obtained for custom hardware accelerators. So I tried to go through the maths to optimize them completely mathematically, but the number of variables blows up as we approach greater than 3 to 4 layers causing very long time for the optimization. Therefore, I had to switch to the current approach. Still I believe this approach can be explored further.

V. RESULTS

We evaluated PolyFuse against both a standard layer-by-layer mapping (Naive) and a state-of-the-art mapper (DeepFrack) for the AlexNet architecture on the Simba accelerator.

A. Energy Consumption

As shown in our evaluations, PolyFuse identifies a strictly optimal dataflow mapping that outperforms both baselines. The total system energy consumption was measured as follows:

- **Naive (layer-by-layer):** 5.76 mJ (5,762,310,000 pJ)
- **DeepFrack:** 3.88 mJ (3,875,573,794 pJ)
- **PolyFuse:** 3.75 mJ (3,747,816,226 pJ)

PolyFuse achieved a **1.54x energy efficiency improvement** over the Naive baseline.

B. Exploration Time

A major drawback of existing DSE tools is their prohibitively long end-to-end search times. Mappers like DeepFrack require a massive, brute-force benchmarking phase to simulate all possible tile sizes on the hardware model before scheduling can even begin. As reported in the DeepFrack paper, this end-to-end process takes approximately **6 hours**.

In stark contrast, PolyFuse entirely eliminates the brute-force benchmarking phase by utilizing our analytical tile calculation to mathematically determine the physical tile bounds. Combined with our $O(N^2)$ dynamic programming partitioner, PolyFuse completes the identical end-to-end mapping task for AlexNet in just **5 to 7 minutes**. This represents a **30–40x reduction** in overall search time.

REFERENCES

- [1] A. Parashar et al., "Timeloop: A systematic approach to dnn accelerator evaluation," in *2019 IEEE International Symposium on Performance Analysis of Systems and Software (ISPASS)*. IEEE, 2019, pp. 304–315.
- [2] Y. N. Wu et al., "Accelergy: An architecture-level energy estimation methodology for accelerator designs," in *2019 IEEE/ACM International Conference on Computer-Aided Design (ICCAD)*. IEEE, 2019.
- [3] M. Gilbert et al., "Looptree: Enabling exploration of fused-layer dataflow accelerators," in *2023 IEEE International Symposium on Performance Analysis of Systems and Software (ISPASS)*. IEEE, 2023, pp. 316–318.
- [4] T. Glint, M. Pechimuthu, and J. Mekie, "DeepFrack: A Comprehensive Framework for Layer Fusion, Face Tiling, and Efficient Mapping in DNN Hardware Accelerators," in *2024 Design, Automation & Test in Europe Conference (DATE 2024)*. IEEE, 2024.
- [5] U. Bondhugula, A. Hartono, J. Ramanujam, and P. Sadayappan, "A practical automatic polyhedral parallelizer and locality optimizer," in *Proceedings of the 29th ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI)*, 2008, pp. 101–113.